OpenZeppelin | security

# ZKsync OS Audit


September 2, 2025

# Table of Contents

# Summary

| | |
|---|---|
| Timeline | From 2025-06-09<br>To 2025-06-20 |
| Languages | Rust |

| | |
|---|---|
| Total Issues | 22 (16 resolved, 1 partially resolved) --> |
| Critical Severity Issues | 3 (3 resolved) |
| High Severity Issues | 1 (1 resolved) |
| Medium Severity Issues | 3 (1 resolved) |
| Low Severity Issues | 9 (8 resolved) |
| Notes & Additional Information | 6 (3 resolved, 1 partially resolved) |
| Client Reported Issues | 0 (0 resolved) |

# Scope

In the first part of the engagement, we performed an assessment of the matter-labs/zksync-os repository at the 96d9d37 commit.

In scope were the Rust files under the following directories:

```
.
├── basic_bootloader
│   └── src
│       └── bootloader
│           ├── account_models
│           └── transaction
├── basic_system
│   └── src
│       ├── system_functions
│       └── system_implementation
│           ├── flat_storage_model
│           ├── memory
│           └── system
└── system_hooks
    └── src
```

The files `abstract_account.rs` and `contract.rs` under `basic_bootloader/src/bootloader/account_models` were left out of scope because account abstraction is not yet supported.

In the second part, we performed an assessment of the matter-labs/zksync-os repository at the 0563213 commit.

In scope were the Rust files under the following directories:

```
.
evm_interpreter
└── src
    └── instructions
zk_ee
├── src
│   ├── common_structs
│   │   └── history_map
│   ├── common_traits
│   ├── kv_markers
│   ├── memory
│   ├── oracle
│   ├── reference_implementations
```

```
│   ├── system
│   │   ├── errors
│   │   └── execution_environment
│   ├── system_io_oracle
│   ├── types_config
│   └── utils
│       └── convenience
zksync_os_runner
└── src
oracle_provider
└── src
forward_system
└── src
    ├── run
    └── system
```

# Audit Scope

The audit was performed on the matter-labs/zksync-os repository at the 5e69d44 commit. The scope encompasses both scopes of the first and second part of the assessment listed above.

# Approach

Due to the significant size of the codebase, the engagement was divided into 3 phases. In the first two phases, a security assessment was conducted on separate scopes, in which the primary goal was to become familiar with the overall architecture and identify critical components of the ZKsync OS codebase, laying the ground for phase 3, in which an audit was be conducted on both the scopes from the initial assessments.

The assessments focus on understanding the technology stack and overall system architecture, emphasizing critical security areas to identify high-level design flaws, structural weaknesses, and inherited vulnerabilities. The objective is to evaluate the protocol's overall security posture and provide actionable recommendations to mitigate identified risks.

In the first phase, our assessment covered three foundational components:

- The **Bootloader**, which orchestrates how transactions are analyzed and dispatched.
- The **System** module, which provides execution environments with access to storage, memory, and oracles.

- **System hooks**, the framework for executing precompiles and other ZKsync-specific contracts.

This second phase focused on several key components of the ZKsync OS:

- The general **Execution Environment (EE) Framework**, which defines the core structure for all execution environments.
- The **EVM Execution Environment**, which is ZKsync's specific implementation of the EVM.
- The **Oracle Provider**, the component that supplies data during execution.
- The **ZKsync OS Runner**, which executes the ZKsync OS in a simulator for proof generation.
- The **Forwarder System**, the module executes ZKsync OS in execution mode.

# System Overview

ZKsync OS is the core execution framework of the ZKsync network. It also generates zero-knowledge proofs attesting to the correctness of these state transitions, enabling secure and scalable settlement on Ethereum. Its primary function is to run batches of transactions and calculate the new state of the blockchain as a whole. The system takes in transactions and initial data and produces a new state for the entire network.

The system was designed to fulfill the following goals:

- **Ethereum Compatibility:** It must be type 2 fully Ethereum Virtual Machine (EVM) equivalent. This is a flagship feature as it allows developers to seamlessly bring their existing Ethereum applications into ZKsync.
- **High Performance:** It will offer high transaction throughput, making extremely low transaction fees a possibility.
- **Customizability:** The architecture is modular, i.e., it can be easily configured and extended. This allows it to support different virtual machines, e.g., backwards compatibility with EraVM or Wasm VM to allow for smart contracts beyond Solidity. However, in the current version, only the EVM is supported, while the infrastructure to support more VMs is already implemented.

ZKsync OS operates in two modes:

1. **Forward Mode:** It is the live run mode used by the network sequencer, the component responsible for ordering transactions. It is performance-optimized for high-throughput processing.
2. **Proof Mode:** The proof mode is used to generate proofs that all transactions were processed correctly. The resulting proof is subsequently published to a settlement layer, such as Ethereum, to finalize the transactions and inherit its security guarantees.

To cover the cost of these two modes, ZKsync OS implements a dual resource accounting mechanism. It distinguishes between the computational cost of running a transaction (what users pay for in gas) and the cost of generating its proof. By recording both, the system ensures that its economic model is sustainable and accurately reflects the resources used.

# Bootloader

The Bootloader is the orchestration component of all operations in ZKsync OS. It initializes the system and manages the entire life cycle of a block, starting from processing the first transaction to finishing the content of the block.

The bootloader's main responsibility is to execute a loop that processes a transaction at a time:

1. It begins by initializing the system and reading the new block context.
2. It then reads, verifies, and executes each transaction sequentially.
3. After processing all transactions, it finishes the block by generating a block header that contains the result of the execution.

### Transaction Processing

The main function of the bootloader is to handle transactions. It is handled slightly differently for normal Layer 2 transactions and for those transactions that have been transferred from Layer 1.

- **Validation:** The bootloader validates a series of conditions before executing any code. It checks the transaction nonce, verifies the sender's signature, and checks the sender's account has enough funds to cover the maximum possible cost of the transaction. A critical part of this step is also verifying that the transaction can pay for its share of pubdata - data that must be published to the settlement layer to guarantee data availability.
- **Execution:** Once a transaction has been correctly validated, the bootloader will proceed to run it through the `runner` component. The `runner` is a coordinator that manages the call stack and transfers requests to the appropriate execution environment. This design enables complex interactions, such as a contract in one environment calling another contract in a different one.
- **Refunding:** After execution, the bootloader ensures that any remaining balance from the pre-paid fee is refunded to the user.

The system is designed for broad compatibility. The bootloader supports various types of transactions, including legacy Ethereum transactions, EIP-1559 transactions, native ZKsync EIP-712 transactions, and L1 -> L2 messages initiated from the Ethereum mainnet. L1 -> L2 transactions are given special priority, as they are already considered secure by L1 and do not need to follow the standard validation procedure.

# System

The System serves as the intermediary between high-level transaction execution and low-level resource management. The System is modularly structured to support two principal operation modes: a "forward running" mode used by the sequencer, and a "proving" mode used to generate validity proofs.

It is passed to each Execution Environment and provides them with access to three important modules: I/O, Memory, and Oracles. Note that Oracles are discussed in the designated section in detail in Phase 2 of the assessment.

- **I/O:** The I/O Subsystem offers an abstraction of all interaction with the state of the blockchain. It is a key-value store where data is read by address and key. The minimum implementation involves a main persistent storage along with several temporary storages for things like logs and events, which are flushed after every transaction.
- **Memory Subsystem:** This subsystem provides a memory per execution environment. This guarantees that when a contract is executing, it has its own memory area that cannot be accessed or touched by other contracts. When an execution frame finishes, data that needs to be returned is copied to another specialized area so that it can be protected from being overwritten.
- **Oracles:** Oracles are the interface through which the system accesses outside information and will be discussed in detail in Phase 2 of the assessment. Note that the system employs two different oracles, depending on if the system is running in proving mode or execution mode. In proving mode, the system does have the responsibility of verifying the data it obtains. For instance, bytecode received from an oracle is re-hashed and verified before being used in the proving environment.

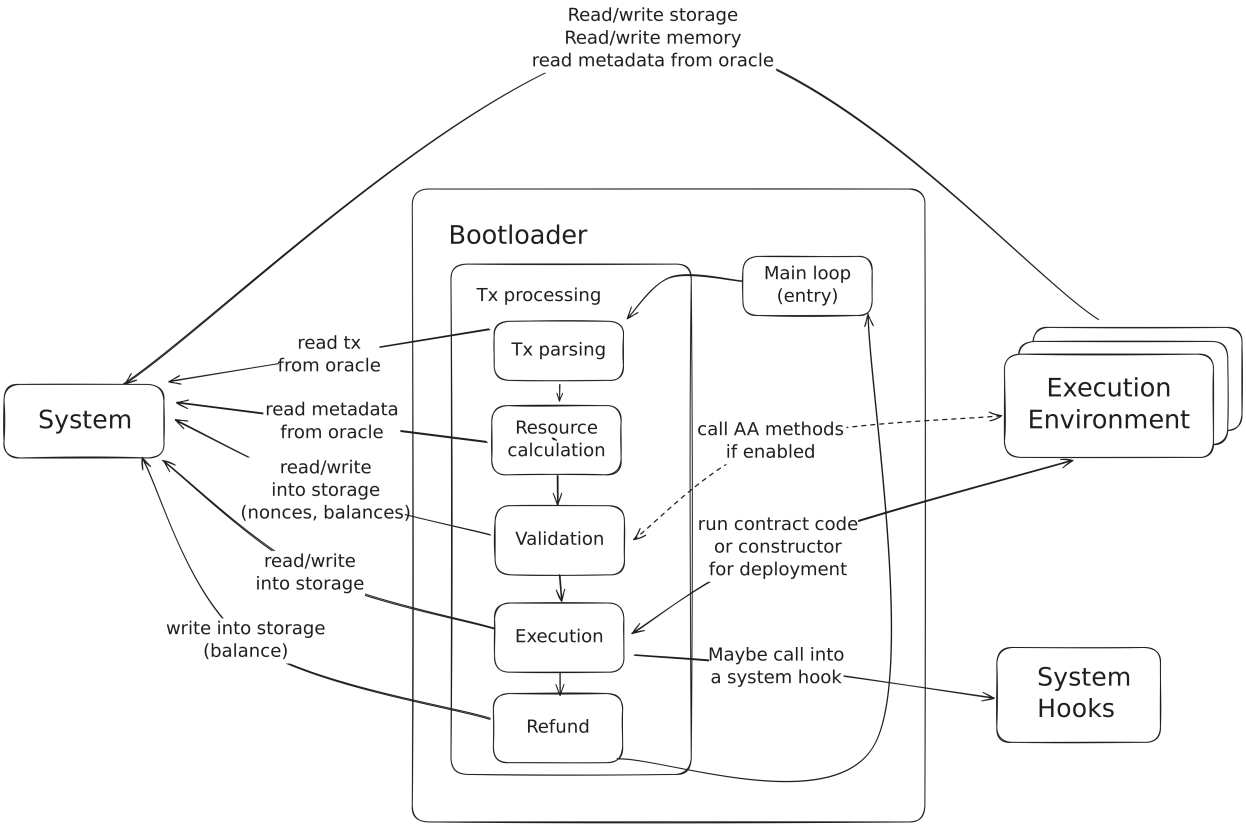
# System Hooks

System hooks are a unique type of function with pre-specified system addresses. When a call is invoked on such an address, ZKsync OS intercepts it and executes a native, hard-coded function instead of EVM bytecode. It plays a dual role:

- **Using Precompiles:** Most popular cryptographic functions (like `ecrecover`, `sha256`, etc.) are expensive to perform computationally within a virtual machine. System hooks enable highly optimized, native implementations of these operations which are far less expensive and quicker to execute. As a compatibility measure with Ethereum, these precompile hooks are installed at the same locations they exist in the EVM.

- **System Contracts:** Specialized contracts that carry out fundamental protocol operations. Some examples are:
  - **L1 Messenger:** A specific hook that enables contracts to securely message back to the Layer 1 settlement layer.
  - **L2 Base Token:** A hook that contains functionality for the native ZKsync token, like withdrawals.
  - **Contract Deployer:** A one-time hook that can only be called by a special system address, deployable to any address. This is intended to be used in governance-approved protocol upgrades.

A rough visualization of a transaction's lifecycle can be seen in the image below:




# Execution Environment (EE) Framework

The Execution Environment (EE) Framework is the abstraction framework of ZKsync OS for supporting multiple virtual machines within a single system. The EE Framework makes ZkSync capable of having distinct execution environments like EVM, EraVM, and WebAssembly supported uniformly with the same interface.

The EE model provides a standard interface that each execution environment must implement, launch parameters, preemption points, and continuation procedures. Each EE operates with an execution loop until a preemption point (external call, deployment, or termination), returns control to the bootloader, and resumes after the bootloader has handled the request.

The architecture supports call modifiers (static, delegate, constructor), resource handling with variable gas conversion ratios per EE, and deployment preparation with address derivation.

## EVM Execution Environment

The EVM Execution Environment provides full native EVM equivalence in ZKsync OS, with a complete EVM interpreter implemented that maintains compatibility with existing Ethereum tooling and contracts.

The EVM interpreter is organized in the `evm_interpreter` crate and provides a complete EVM implementation including stack-based execution, memory management, and full opcode support. The interpreter is implemented as a component of ZkSync's dual resource accounting system, billing both EVM gas (translated into ergs) and native resources for proof expenses.

This Ethereum compatibility enables live Ethereum contracts to run unchanged, supporting the developer experience, and testing using the Ethereum Foundation test suite for complete compatibility testing.

However, several known divergences from Ethereum remain:

- Additional **pubdata** fees that may impact keyless transactions.
- Deployments by contract don't fail even when the target address contains some pre-existing storage (in situations with a zero nonce and zero code).
- Nonces are represented as a 32-bit integer, which breaks the larger nonce constraint as defined by **EIP-2681**.
- The **DIFFICULTY** opcode (**PREVRANDAO**) is unsupported and returns a faked value of 0.

## Oracle Provider

Oracle Provider is the bridge between ZKsync OS execution and RISC-V proving environment that provides non-deterministic input by injecting external information into the proving system with deterministic execution.

Oracle provider provides two major components: `ZkEENonDeterminismSource` to process query processors and `BasicZkEEOracleWrapper` for adapting ZKsync OS oracles to the non-determinism system. The system uses a query-response scheme where the RISC-V environment writes the query arguments and reads responses through dedicated Control and Status Registers (CSR).

The oracle provider governs different kinds of requests like transaction history, storage reads, and block data.

## ZKsync OS Runner

The ZKsync OS Runner is the interface to the RISC-V simulator that executes ZKsync OS binaries for proof generation as well as testing purposes, serving as the go-between between compiled ZKsync OS Runner binary and the RISC-V runtime.

The runner loads ZKsync OS RISC-V binaries and executes them with provided non-determinism sources. It utilizes a specified register convention where the final 256-bit output is stored in RISC-V CSRs and made available as public input for zero-knowledge proofs.

## Forward System

The Forward System utilizes the "forward running mode" of ZKsync OS, which is the runtime environment utilized by the sequencer for live execution of transactions. The forward system provides real-world implementations for executing ZKsync OS in sequencer mode using the normal system allocator and live oracle implementations.

The forward system uses batch execution by `run_batch` in executing several transactions and transaction simulation by `simulate_tx` in single transaction simulation used by `eth_call` and `eth_estimateGas` RPC calls. The system is made consistent with the proving environment through the usage of the same bootloader core logic while providing optimized resource management during live execution.

# Critical Severity

## C-01 Transaction Routing to Unsupported Execution Environment

During transaction processing, whenever the target address is `SPECIAL_ADDRESS_TO_WASM_DEPLOY`, the transaction will be processed as a deployment to the Wasm execution environment.

In the `bootloader::account_model::eoa` module, the `to_ee_type` variable is either assigned the values `ExecutionEnvironmentType::EVM`, `ExecutionEnvironmentType::IWasm`, or `None`. In case the transaction's target address is `SPECIAL_ADDRESS_TO_WASM_DEPLOY`, the value of `to_ee_type` is set to `ExecutionEnvironmentType::IWasm`. Subsequently, the `execute` function will call `process_deployment` with the detected `to_ee_type`, which will revert with an internal error since the `IWasm` execution environment is not supported yet. This will end up returning an error from `run_prepared`, which will cause a `panic` in both the forwarder and prover invocations.

Consequently, this can cause the sequencer to crash, forcing a system restart. Moreover, this attack can be executed deliberately often, potentially cause a DoS, and slowing down the network's execution.

Consider temporarily disabling all logic related to Wasm deployments until the feature is fully supported to prevent this DoS vector. This includes removing the check that identifies transactions targeting `SPECIAL_ADDRESS_TO_WASM_DEPLOY`, as well as the corresponding logic that charges intrinsic gas for such deployment transactions.

***Update:*** *Resolved in pull request #151 at commit 2b49475 and in pull request #214 at commit b3d55d3.*

## C-02 Return Data Buffer May Be Drained

The return data space of smart contracts is represented through the `return_data` buffer of 128 MB, preallocated before the transactions execution starts. Whenever any data is returned from external calls during a transaction, it is copied to that buffer and the space available for

future return data shrinks. In case when there is not enough space in the remaining part of the return data buffer, the code panics. A similar mechanism is used for precompiles execution, where the remaining data buffer part is also split. In this case, however, if there is not enough space for the return data, an undefined behaviour would happen in the `split_at_mut_unchecked` call.

This could be exploited by an attacker, who deploys and executes a smart contract, which performs many external calls, each of which heavily use the return data, until the return data buffer is drained. This could be achieved either by repeatedly calling a user-space program, or the Identity precompile. In the first case, the cost of returning `x` 32-byte words involves a memory expansion, hence requires at least `3x + x^2/512` gas per call, and the second method requires `~3x` gas per call. Both options theoretically require paying at least 3 gas per return data 32-byte word, although due to the redundant, second return data allocation for precompiles, described in more detail another issue in this report, this cost is reduced to only half of this value.

As such, the attack draining the entire return data buffer could be executed by using `~3 * 4 194 304 / 2 < 6.5M` gas (or `< 13M` gas assuming that the double-allocation issue referenced above is fixed), which is below the target transaction gas limit of 18M. As a consequence, because `panic` occurs, it will not be possible to process such a transaction, which in case of `L1->L2` transaction would stop all subsequent `L1->L2` transactions from executing.

Consider setting a hard gas limit on both L2 and `L1->L2` transactions and increasing the space allocated for the return data buffer, so that the described attack is no longer possible with the new limits. Furthermore, consider using the `split_at_mut_checked` function instead of the unsafe `split_at_mut` and `split_at_mut_unchecked` alternatives and handling the `None` value returned in order to prevent panics in the bootloader.

***Update:*** *Resolved in pull request #218 at commit 3cd893a and in pull request #257 at commit 9afe7dc. The Matter Labs team stated:*

> *We ended up using a different approach: We incremented the returndata buffer to 256 MB, this should be enough for worst-case up to ~18M gas. However, we decided not to implement a per-tx max gas limit, as this will be a divergence from EVM (for now). This also puts a limit on L1 transactions, as pointed out. Instead, we decided to handle the out of return memory error as a fatal error (same handling as out of native resource). We believe this state is only reachable by contracts crafted to exploit this, so we accept this formal divergence (which should not be observable in normal usage). As a reminder, when such fatal error is reached at any point of a transaction's execution, the error is*

> *bubbled up to the top-level and the tx execution is reverted (notice, we do not revert the fee payment, to prevent DDoS).*

## C-03 `usize` Arithmetic Can Lead to Non-Determinism and Panics

Throughout the codebase, there are several places where the `usize` type is used. Since the size of this type is architecture-dependent, the usage of it could cause discrepancies in how the code is executed in different environments or could result in panic. The relevant instances are enumerated below: - The `l2_base_token_hook_inner` function uses the `try_into` function to coerce `message_offset` into `usize`. On 64-bit targets, this admits any value up to `2^64 - 1`, but on 32-bit targets values greater than `2^32 - 1` fail the conversion. This results in a discrepancy in the code behaviour on 64-bit and 32-bit target, where the execution continues and results in an error later on on the former and results in an earlier error in the latter. - A similar problem is present within the `system_hooks`, where dynamic `bytes`-parameters parsing is done using `usize` types. In this case, when handling the `sendToL1(bytes)` function, the `length`, extracted from the calldata could be set to a value close to the `u32::MAX` and the subsequent addition will pass on the sequencer (64-bit target) and fail for passing short calldata, whereas the same operation will revert earlier on the prover (32-bit target). Similar problems may appear for other places in the code where checked arithmetic is used for the `usize` type. - When beginning a new transaction, the bootloader calls the `try_begin_next_tx` function. This function processes incoming transactions by rounding the reported byte length up to a machine word boundary (`USIZE_SIZE`) and then trying to iterate over the transaction content. However, on 32-bit targets, where `USIZE_SIZE == 4`, computing `next_tx_len_bytes.next_multiple_of(USIZE_SIZE)` can overflow `usize` for very large inputs, with size close to `u32::MAX`. In release builds, this overflow wraps to 0, so `next_tx_len_usize_words` becomes 0 while the iterator over the actual content is non-empty. As an overflow does not happen on 64-bit target, this causes a discrepancy in how the transaction data is processed on different targets. Furthermore, processing transactions with the size exceeding the length of the allocated buffer may result in a panic as the `try_begin_next_tx` function is expected to succeed.

For long-term stability, consider avoiding architecture-dependent types like `usize` for arithmetic. Refactoring to use fixed-size integers (e.g., `u32` or `u64`) will ensure consistent and predictable results across all environments. Furthermore, consider explicitly rejecting transactions with a content bigger than the maximum allowed size of ~8 MB in order to prevent potential discrepancies in code execution resulting from `usize` arithmetic and panics in the bootloader.

***Update:*** *Resolved in pull request #197 at commit fabf065 and in pull request #215 at commit 80876b3.*

# High Severity

## H-01 Legacy Transactions May Be DoSed by Appended Access Lists

The EVM Cancun specification mentiones several available transaction types. While the EIP-4844 (`BlobTransaction`) type is deliberately not supported in ZKSync OS at the moment, there is a discrepancy between how the `LegacyTransaction` type is handled in both environments.

Specifically, in the EVM Cancun , the access lists are not supported in this type of transaction, but they are still processed in ZKSync OS as the same logic will be applied to them as for other L2 transactions. This results in a discrepancy in how legacy transactions are executed on Ethereum and on ZKSync OS that can lead to two different consequences.

The first one allows users to benefit from access lists, while not paying the native fee for their length in hash calculation. This could be achieved by sending a legacy transaction, but still including an access list in it.

The second consequence also follows from the fact that access lists are not included in a legacy transactions' hashes, which are then used for verifying transactions' signatures. This allows an attacker, who intercepts a valid legacy transaction without access list, to append an arbitrary access list to it and this transaction would still be considered valid, as the data to be signed did not change, since it does not take into account an access list. However, the access list would still be processed, which would cause a victim to lose gas. This could be used to cause any legacy transaction to revert with OOG by spending almost the entire available gas, so that the transaction would still be executed, but would quickly revert, causing a victim to lose funds.

Consider rejecting any legacy transaction that contains an access list, to avoid allowing attackers from manipulating legacy transactions as well as aligning with the EVM Cancun specification.

***Update:*** *Resolved in pull request #154 at commit 1a60d90.*

# Medium Severity

## M-01 Improper Accounting for Transaction and Block Gas Limits

When processing an L2 transaction, it is ensured that `block_gas_limit <= MAX_BLOCK_GAS_LIMIT` and that `tx_gas_limit <= block_gas_limit`. For L1 to L2 transactions, the current gas limit per transaction is set to `72_000_000` and is only checked in L1 contracts.

However, both checks do not take into account used gas in a block. Moreover, there is no check in the `bootloader` that ensures that `block_gas_limit` is not exceeded when adding all used gas of a block's transactions together. Currently, it is only documented that the sum of `gas_used` in a block should not be `0`.

Following the Cancun specs, a transaction's gas limit must not exceed the available gas in a block, which is calculated by subtracting the gas used from the gas limit of a block. Additionally, the gas used in a block must not exceed the block's gas limit.

This improper gas accounting can allow the executor to include blocks with an arbitrary number of transactions that violate the block's gas limit.

To ensure compatibility with Cancun specs, consider accounting for used gas during transactions and ensuring that the transaction gas limit does not exceed the remaining gas in a block, as well as ensuring that the sum of all transaction gas used does not exceed a block's gas limit. Alternatively, since the accounting for used gas is a known `TODO`, consider expanding the comment on line 349, highlighting the missing check while only allowing for one transaction per block until the `TODO` is resolved which will ensure that the gas used is less than or equal to `MAX_BLOCK_GAS_LIMIT`.

Additionally, consider reducing the transaction's gas limit for L1 to L2 transactions. This will prevent users from initiating transactions that are deemed valid by L1 contracts, but will fail to execute on ZKsync OS.

***Update:*** *Acknowledged, not resolved. The Matter Labs team stated:*

> *The pull request #191 enforces block limits, making transactions that overflow them (in this case, block gas limit) invalid. The sequencer will remove it from the final block.*

> *Block gas usage calculation has also been implemented. For L1 transactions, we're considering reducing the limit on L1.*

## M-02 Missing Getter Functions in L2 Base Token Contract

The L2 Base Token was originally implemented in Solidity and has since been migrated to the ZKsync OS environment using Rust off-chain implementation as a hook. In the original implementation, several functions—such as withdrawals, `balanceOf`, and the auto-generated `totalSupply`—were available. These functions are commonly used by external contracts and interfaces to interact with and retrieve information from the token contract. In the current Rust implementation, some of these getter functions are missing, which may lead to inconsistencies when existing or new contracts attempt to interact with the L2 Base Token.

The absence of expected functions like `balanceOf` and `totalSupply` in the Rust implementation may result in broken functionality for contracts or services that rely on them. These omissions can cause integration failures or runtime errors during execution, especially in systems expecting behavior consistent with ERC-20-like tokens.

Consider implementing all public functions from the previous Solidity-based L2 Base Token, including getters like `balanceOf` and `totalSupply`, to ensure backwards compatibility and consistent behavior across environments.

***Update:*** *Acknowledged, will resolve. The Matter Labs team stated:*

> *We are not convinced that this is an issue. This release targets EVM equivalence, and the base token doesn't need to be ERC-20 compliant (like ETH on L1). This system hook is only providing withdrawal functionality. We'll include other methods for backwards compatibility when we migrate ZKsync Era.*

## M-03 Discrepancy With EVM in Return Data Handling During Contract Deployments

The EVM specification mandates that when a contract deployment failed because of incorrect first byte of code or too long code, the return data should be cleared.

However, on ZKsync OS, in such a case, the return data, containing the contract's code is not cleared, then saved as the return data from deployment and finally propagated to the calling contract. It could cause unexpected behaviour on the calling contract's side, which could

expect an error information, but would instead receive a huge return data, which would trigger costly memory expansion and could result in OOG errors.

For example, in case of the OpenZeppelin's `Create2` library, this behaviour could cause an unexpected revert when copying the revert message from the `create2` call.

Consider following the EVM specification and clearing the contract's code from return data in case where an error happens during the validation of the code to be deployed.

***Update:*** *Resolved in pull request #226 at commit d8a3733.*

# Low Severity

## L-01 Inconsistent Dirty Bits Check on L1 Receiver Address

In `l2_base_token`, the `WITHDRAW_SELECTOR` path validates that the L1 receiver address has no dirty bits, while the `WITHDRAW_WITH_MESSAGE_SELECTOR` path omits this check and slices 20 bytes directly; in Solidity this would revert on dirty bits, leading to inconsistent behavior between the two paths.

Consider aligning the address validation with Solidity by enforcing the dirty-bit check for addresses uniformly across both withdraw paths so non-zero upper bytes cause a revert.

***Update:*** *Resolved in pull request #192 at commit b184113.*

## L-02 Native Gas Cost Anomaly for `PUSH` Opcodes

The native gas costs for the `PUSH` family of opcodes are expected to be monotonic. This means the cost to push N+1 bytes to the stack should be greater than or equal to the cost of pushing N bytes. The defined constants for native gas costs violate this expectation. Specifically, `PUSH15_NATIVE_COST` is 240, while `PUSH16_NATIVE_COST` is only 210. This makes it cheaper to push 16 bytes than it is to push 15 bytes.

Consider reviewing and correcting the entire `PUSH<N>_NATIVE_COST` table to ensure the values increase monotonically.

***Update:*** *Resolved in pull request #228 at commit 511b5df.*

## L-03 Discrepancy in Call-Stack Handling and Error Ordering in `external_call_before_vm`

In EVM Cancun, an external call first verifies that the current depth does not exceed 1 024 before attempting any state-changing action such as transferring value; if the limit is exceeded the call fails immediately and no Ether moves. The ZKsync OS implementation diverges: within `external_call_before_vm`, value is transferred before the depth check, and the early-exit branch for externally owned accounts (EOAs) returns success without ever evaluating `self.callstack_height > 1024`.

Because of this ordering, a call that is already deeper than 1 024 frames but targets an EOA will still move funds and be reported as successful, whereas the EVM would fail with `StackDepthLimitError` and revert the transfer. For non-EOA targets the function instead returns `OutOfNativeResources` after the premature transfer, creating a second kind of mismatch with EVM behaviour.

While it is not possible to exploit the issue with the current configuration, this may change with a chain upgrade, potentially making the issue exploitable.

Consider reordering the logic so that every outbound call—regardless of target type—checks `self.callstack_height` before any value transfer and before the EOA early-return, thereby matching EVM semantics and producing consistent error codes.

***Update:*** *Resolved in pull request #184. The Matter Labs team stated:*

> *This has been fixed in pull request #184. This is a big simplification of the runner, but the relevant change for this issue is that now this check is performed more consistently and in the right order. Now this checks are part of the EVM `before_executing_frame` function, which is also called when in NoEE (call to EOA), as EVM is the "default" behaviour for EOA.*

*The OpenZeppelin team stated:*

> *Although the issue is no longer part of the codebase due to the changes in the linked pull request, we do not consider the changes in this pull request as part of the final audited commit due to significant changes, including out-of-scope changes.*

## L-04 Inconsistent Contract Detection

The `is_contract` function considers an address a contract if either `unpadded_code_len` or `artifacts_len` is greater than zero, while `is_eoa` flag checks only for zero bytecode length.

Consider unifying the logic for contract detection to avoid inconsistent behavior across the system.

***Update:*** *Resolved in pull request #184. The Matter Labs team stated:*

> *This is no longer a problem. Pull request #184 introduced a simplification of the runner. The PR is quite large, but you can see that the function `call_execute_callee_frame` will now do an early return on NoEE.*

*The OpenZeppelin team stated:*

> *Although the issue is no longer part of the codebase due to the changes in the linked pull request, we do not consider the changes in this pull request as part of the final audited commit due to significant changes, including out-of-scope changes.*

## L-05 Inconsistency in Contract Deployment with Respect to EVM

The EVM specification for the Cancun fork mandates that in case when insufficient amount of gas has been provided for contract deployment, the transaction should be rejected during the validation phase. The required amount of gas includes both the base creation cost and the init code cost.

However, on ZKsync OS, only the init code cost is taken into account during the validation phase and the base creation cost is charged later on, in the execution phase. As a result, a transaction providing insufficient gas to cover the base creation cost, would be rejected on Ethereum during the validation phase, but would be processed and reverted during the execution phase on ZKsync OS.

Consider including the base creation cost in the transaction validation phase in order to maintain compatibility with EVM specification.

***Update:*** *Resolved in pull request #196 at commit 56e4446.*

## L-06 Double Return Data Allocation for Precompiles

Whenever external calls to precompiles complete, the return data is copied to the return data buffer, allocated before the transactions execute.

However, during the actual call to precompiles, the return data is already copied to the return data buffer, hence the subsequent return buffer allocation is not necessary.

Consider removing redundant return data allocation for the precompiles.

***Update:*** *Resolved in pull request #193 at commit efeaf3a.*

## L-07 Inconsistency in `coinbase` Rewards Handling with Respect to EVM

According to the EVM specifications, whenever a contract is created and `selfdestruct`ed in the same transaction, the contract is not immediately deleted, but marked for deletion which actually happens at the end of the transaction. The deletion of an account involves the removal of its storage and setting it to `None`. The latter operation effectively removes the entire balance of an account.

The accounts deletion happens at the very end of the transaction, notably, after the transaction reward is transferred to the `coinbase` address. It means that if a contract, which is created and deleted in the same transaction is set as the `coinbase` address, the reward received to that address is permanently burnt.

However, on ZKsync OS, the actual reward transfer happens after deletion of accounts. It means that the reward is never burnt and the contract which was destructed is initialised once again at the end of transaction processing.

Consider performing accounts deletion after processing the `coinbase` rewards, or documenting the current design choice of processing it before transferring the reward.

***Update:*** *Resolved in pull request #229 at commit 4a885c7.*

## L-08 Out of Sync Transaction Counters

The system's current approach to tracking transaction numbers within a block is inconsistent across different components. The primary transaction counter within the `io_subsystem` is

incremented at the completion of a transaction, which correctly starts the numbering at index zero. However, internal data caches, particularly storage and account caches, use their own separate counters that are instead incremented at the beginning of a new transaction [1] [2].

This discrepancy results in different parts of the system holding different values for the "current" transaction number, which can lead to confusion and is a source of potential bugs.

To improve clarity and reliability, consider adopting a single, uniform method for counting transactions, centralizing this logic within the `io_subsystem`, or adopting the same counting logic throughout the system.

***Update:*** *Resolved in pull request #231 at commit 474c6ff. The Matter Labs team stated:*

> *Acknowledged, we went for the simpler option (being consistent in when we update these counters). We'll probably unify the counter in a later release.*

## L-09 Misusage of the `Result` Type

The `result` variable in the EVM interpreter is inferred to the standard `Result<(), ExitCode>` type. This type can be misleading, as not all `Exitcode`s are errors. The issue arises because the `Result` type implies that the `ExitCode` variant is always an error, although it is used for all exit reasons, including success conditions. This ambiguity introduces the risk that future maintainers might use the `?` operator on this value, causing a success state to be incorrectly propagated as a critical failure.

Consider replacing the standard `Result<(), ExitCode>` with the `Option<ExitCode>` type. Alternatively, consider using a custom `enum`, for example, `EVMInterpreterResult`. This would force the caller to use a `match` statement to handle the different exit conditions explicitly, preventing confusion between success and error states and ensuring the `?` operator cannot be misused.

***Update:*** *Acknowledged, will resolve. The Matter Labs team stated:*

> *We agree to restructure this return type for the next version.*

# Notes & Additional Information

## N-01 Discrepancy Between Code and Comment Description

The `flush_tx` function is used to finish the current transaction execution. According to the function's comment, it should also return execution stats. However, in case of success, the function always returns `Ok(0)`.

Consider returning transaction stats instead of `Ok(0)` to match the code functionality to the comment.

***Update:*** *Resolved in pull request #237 at commit 4bc28c4.*

## N-02 Naming Suggestions

Throughout the codebase, some instances were identified that could benefit from renaming:

- `start_global_frame` can be renamed to `start_frame`, since the `start_global_frame` is used more than once throughout a transaction, whereas the name gives the impression that a frame is started once in a transaction. Consequently, `finish_global_frame` can be renamed to `finish_frame` to match its counterpart.
- `tx` can be renamed to `expected` or `expected_from`.
- `io` can be renamed to `storage`.
- `diff` can be renamed to `abs_diff`, making the intention of the function clearer.
- `BasicBootloaderForwardSimulationConfig` can be renamed to `BasicBootloaderForwardConfig`.

Consider renaming the instances mentioned above for improved readability and clarity.

***Update:*** *Acknowledged, not resolved. The Matter Labs team stated:*

> *This one I don't think we'll apply. I personally don't think these renamings improve readability much, we'll improve in-code documentation for that.*

## N-03 Unused Enum Variants in `ExitCode`

In the `ExitCode` enum, which defines possible outcomes from the EVM interpreter, several variants are declared but not used within the codebase. These include `OutOfFund`, `CallTooDeep`, and `FatalExternalError`.

Consider removing the unused variants from the `ExitCode` enum if they are not required, or implementing their usage if they represent valid and necessary interpreter states.

***Update:*** *Resolved in pull request #212 at commit 6f1f08d.*

## N-04 Typographical Errors

The following is a list of identified typographical errors throughout the codebase:

- "Our" should be "Out".
- "3th" should be "3rd".
- `in_constructor` should be `is_constructor`.
- "beoynd" should be "beyond".
- "rocessed" should be "processed".
- "in" should be "In".
- `"STORE"` should be `"TSTORE"`.
- "Cleae" should be "Clear".
- "caller" should be "callee". Alternatively, "to" should be "from".
- "that does not that" should be "that does not track".

Consider fixing the instances listed above in order to improve the clarity of the codebase.

***Update:*** *Partially resolved in pull request #237 at commit 501efd1.*

## N-05 Usage of Unstable Features

In Rust, unstable features are experimental APIs that are only available on the nightly compiler and are subject to change or removal without notice. They are typically used for testing and development of new language capabilities before stabilization. Using these features in production code can lead to maintenance challenges, as future compiler updates may break the build or alter behavior.

In the codebase, several function calls rely on unstable features: - In the `HooksStorage` implementation block, the `new_in` function calls `BTreeMap::new_in`, which is unstable. -

In the `BasicBootloader` implementation block, the `run_prepared` function calls `Box::new_uninit_slice_in`, which is unstable.

Consider replacing these unstable feature calls with stable alternatives or refactoring the implementation to avoid nightly-only APIs. If the functionality is essential and no stable API is available, evaluate whether enabling the relevant feature gates is acceptable for your project's stability requirements, and document this decision clearly for future maintainers.

***Update:*** *Acknowledged, will resolve. The Matter Labs team stated:*

> *We'll stick to allocator API, precise version of compiler will be documented and a reproducibility pipeline will be available.*

### N-06 Inconsistent Initialization of Zero-Value `Ergs`

Throughout the codebase, many instances inconsistently create zero-value `Ergs` objects, using both `Ergs::empty()` and `Ergs(0)` interchangeably.

Consider standardizing on the `Ergs::empty()` constructor for all zero-value initializations. This would improve code consistency and align with existing patterns already used for similar types, such as `Native::empty`.

***Update:*** *Resolved in pull request #237 at commit e0cb133.*

# Recommendations

## Phase 1

This section outlines key recommendations based on our initial security assessment of the codebase.

While the codebase includes numerous tests, helpful overview documentation, and a generally well-organized structure, it still presents several quality concerns that could impact security, readability, and maintainability. As this is a high-level assessment rather than an exhaustive audit, the following points are intended to provide actionable advice for enhancing the system's security and code quality.

## Arithmetic on `usize` Can Lead to Halting Block Finalization

The ZKsync OS system is designed to run on different architectures: the executor typically runs on a 64-bit machine, while the prover is designed for a 32-bit environment. In Rust, the data type `usize` represents memory-sized integers. This means `usize` is 64 bits on a 64-bit machine and 32 bits on a 32-bit machine.

Using a platform-dependent type like `usize` for deterministic arithmetic can lead to divergence between executor and prover since each is running on a different architecture. A calculation that uses checked arithmetics on `usize` can pass on the 64-bit executor but fail on the 32-bit prover. On the other hand, if an unchecked arithmetic operation overflows on the 32-bit prover, it does not necessarily overflow on the executor. When this happens, the executor will consider a block valid, but the prover will be unable to generate a proof for it, effectively halting the finalization of blocks on L1.

Consider replacing `usize` with fixed-size integer types to ensure that all calculations produce the same result regardless of the underlying architecture, preventing divergences between the executor and the prover.

## Inconsistent System Configuration and Compilation Flags

ZKsync OS must support different behaviors for its two primary environments: the live execution mode and the proving mode. This is currently managed using conditional compilation flags that include or exclude code based on the target architecture. The method for detecting the target environment is inconsistent across the codebase. Different modules use different flags, leading to a confusing and error-prone setup. For example:

- `cycle_marker` uses `#[cfg(target_arch = "riscv32")]` and `#[cfg(not(target_arch = "riscv32"))]`.
- `crypto::ark_ff_delegation::biginteger` uses `#[cfg(target_arch = "x86_64")]` and `#[cfg(not(target_arch = "x86_64"))]`.
- In `basic_bootloader::bootloader` on lines 102 to 126 we use `#[cfg(target_pointer_width = "32")]` and `#[cfg(target_pointer_width = "64")]` to detect the architecture, and we check for a third option to fail compiling if none of the architectures was detected. This behavior is not consistent, as seen in another function.

- `basic_bootloader::bootloader::result_keeper` uses different implementations for `ResultKeeperExt`, relying on the developer to use the correct implementation.

Furthermore, the `BasicBootloaderForwardSimulationConfig` struct has the same configuration values as `BasicBootloaderProvingExecutionConfig` which might be confusing without further documentation. Although the current system is assumed to have the Account Abstraction feature disabled, there is no implementation for `BasicBootloaderExecutionConfig` where the `AA_ENABLED` field is set to `false`.

Additionally, the `FlatTreeWithAccountsUnderHashesStorageModel` struct includes a `PROOF_ENV` field which is a boolean type, used to define whether the system is in proof mode or not.

This ad-hoc approach increases the risk of misconfiguration where, for example, a developer might add a new feature for one environment but forget to provide the alternative implementation for the other.

Standardize the approach for managing environment-specific configurations. A single, unified feature flag, such as `#[cfg(feature = "executor|prover")]`, should be used consistently across the entire project to distinguish between the execution and proving environments. Additionally, review configuration structs to ensure their names accurately reflect their function and that all necessary permutations are available when needed.

# Discrepancy to EVM

In standard Ethereum (EVM), the `coinbase` address receives its fees immediately after each transaction is successfully processed within a block. In ZKsync OS, however, all transaction fees are first collected in a temporary `BOOTLOADER_FORMAL_ADDRESS`. The funds accumulate there for the duration of the block processing and are only transferred to the final `coinbase` address in a single transaction at the very end of the block. While this is done to maintain compatibility with ZKsync Era's Account Abstraction, it represents a deviation from the EVM's execution model.

To better align with EVM equivalence, consider modifying the fee distribution logic to transfer fees to the `coinbase` address after each individual transaction. Alternatively, the divergence from the EVM standard should be clearly documented for developers and users of the system.

## Magic Values

The usage of undocumented magic values in the codebase can be confusing for readers. Consider documenting the meaning of these values and how they were calculated or defined to enhance readability.

## Usage of `unwrap`, `expect`, and `panic!`

In Rust, methods like `.unwrap()`, `.expect()`, and the `panic!` macro are designed to halt execution immediately when an unexpected state is reached. This is an unrecoverable error that will crash the running program. The codebase contains numerous occurrences of `.unwrap()`, `.expect()`, and `panic!`. While these are appropriate for tests or truly unrecoverable situations, their use in transaction processing logic is dangerous. A specially crafted transaction that triggers a panic could crash the entire sequencer or prover, causing a denial-of-service vulnerability where no new blocks can be processed.

Consider refactoring the codebase to eliminate panics from all core transaction processing and state transition paths. Errors should be propagated and handled gracefully, allowing a transaction to fail and its state changes to be reverted without crashing the entire system.

## Documentation

The project includes high-level design documentation that are helpful for gaining a general understanding of the system. However, secure and maintainable code also relies on detailed inline documentation that explains complex logic at the implementation level.

Most components, complex algorithms, and low-level modules within the codebase lack sufficient inline documentation. For instance, design decisions that deviate from standard EVM must be documented where they are implemented. This lack of context makes the code more difficult to review, harder for new developers to contribute to safely, and increases the risk of introducing bugs during future modifications.

While extensive documentation for a codebase of this size is a significant undertaking, consider prioritizing efforts on public entry-point functions for all critical modules to improve clarity, maintainability, and security.

## Unresolved TODOs

While some TODOs highlight missing features which can be changed or added later, others present potential risks, as they may lead to misuse, errors, or vulnerabilities if not properly addressed.

Consider addressing critical TODOs to prevent the system from failing.

# Phase 2

The second phase of the assessment focused on critical execution and proving components of ZKsync OS, including the Execution Environment framework, the EVM interpreter, the Oracle Provider, the ZKsync OS Runner, and the Forward System. Several recommendations made in Phase 1 - such as improvements to input validation, clarifying assumptions around invariants, and enhancing documentation for unsafe code - remain relevant in this phase as well. Below, we outline a new recommendation, while also noting that previously suggested improvements continue to apply.

## Platform-Independent Bounds for Resource Parameters

As noted in Phase 1, values of type `usize` must be carefully handled when casting to `u32`, particularly across the architecture boundary between the 64-bit forward system and the 32-bit prover.

For example, in the oracle provider code, `new_iterator.len()` returns a `usize` and may exceed the 32-bit limit. Casting without a bound check risks truncation on 64-bit platforms, potentially leading to inconsistent witness generation or prover/verifier desynchronization. Similarly, the prover uses `usize` for cycle tracking but is constrained by a 32-bit architecture. Without an explicit cap, long-running execution paths could exceed the prover's limits (e.g., `2^32 - 1`), leading to overflows or invalid proofs.

Consider adding explicit bounds before downcasting or relying on architecture-constrained values to ensure deterministic behavior across both execution and proving environments.

# Conclusion

ZKsync OS represents a significant evolution of ZKsync's core execution framework, designed to replace the network's current version. This next-generation system introduces a more unified architecture by migrating key components, such as the bootloader and precompiled contracts, from Yul-Assembly to a more maintainable and testable Rust codebase. The most notable architectural improvement is its modular support for multiple Execution Environments (EEs). This design not only preserves compatibility with the existing EraVM but also paves the way for full EVM equivalence and the future integration of a WasmVM, enabling smart contracts to be written in a variety of programming languages.

Both the first and the second phase of this multi-phased engagement revealed a solid and well-considered system design. Our recommendations focus on further enhancing code quality and formalizing system configurations to ensure predictable behavior and improve the development experience.

During the final audit several medium, high, and critical issues were identified and further recommendations were provided for improvement.

We thank the Matter Labs team for their collaboration and responsiveness throughout this engagement, which was supported by clear and adequate documentation.